



Experiment No: 1

Student Name: Manasvi sharma

UID:24BCS10527

Branch: BE.CSE

Section/Group:24BCS_TPP 615 (A)

Semester:5th

Date of Performance: 03/08/2026

Subject Name: Competitive Coding – II.

Subject Code: 24CSP-305

1.(a) LeetCode Problem: 66. Plus One

1.Aim: To design and implement an efficient algorithm to increment a large integer represented as an array of digits by one and return the resulting array.

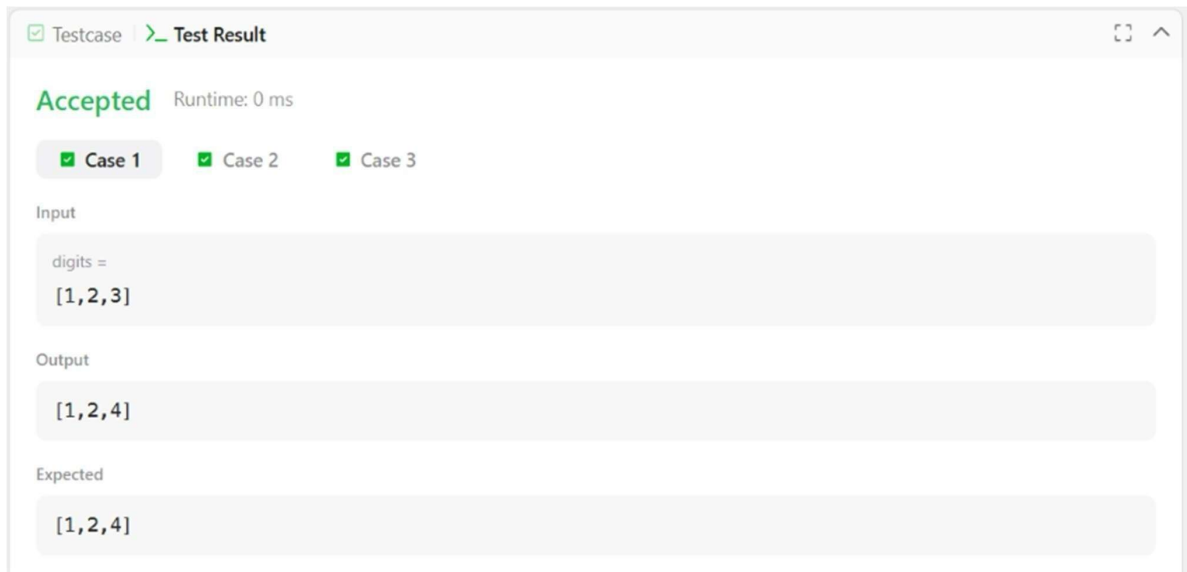
2. Objective:

1. To understand array manipulation techniques.
2. To handle carry propagation while performing arithmetic operations on arrays.
3. To analyze time and space complexity of the solution.
4. To validate the solution using test cases and edge cases.

3./Implementation/Code:

```
class Solution { public:
    vector<int> plusOne(vector<int>& digits) { for
        (int i = digits.size() - 1; i >= 0; i--) {
            if (digits[i] < 9) {
                1,digits[i]++;
                return digits;
            }
            digits[i] = 0;
        }
        digits.insert(digits.begin(), 1); return
        digits;
    }
};
```

4.Output:



5.Learning outcomes (What I have learnt):

- 1.Learned how to traverse an array from right to left.
- 2.Understood the concept of carry propagation during addition.
- 3.Learned how to solve problems involving arrays without converting them into integers.
- 4.Understood how to handle edge cases such as numbers containing all 9s.
- 5.Improved problem-solving skills using efficient array manipulation techniques.

1.(b) LeetCode Problem:18.4Sum

1.Aim:

To design and implement an efficient algorithm to find all unique quadruplets in an array whose sum equals the target value.

2.Objective:

- 1.To apply sorting and two-pointer techniques for array-based problems.
- 2.To efficiently identify unique quadruplets without duplicates.
- 3.To analyze time and space complexity of the solution.
- 4.To validate the solution using test cases and edge cases

3.Implementation/Code:

```
class Solution {
public:
vector<vector<int>>>fourSum(vector<int>& nums, int target) {

    sort(nums.begin(),nums.end());
    vector<vector<int>>> ans;
    int n = nums.size();

    for(int i = 0; i < n - 3; i++) {

        if(i>0&&nums[i]==nums[i - 1])
            continue;

        for(intj=i+1;j<n-2;j++) {

            if(j>i+1&&nums[j]== nums[j - 1])
                continue;

            int left = j + 1;
            int right = n - 1;
            while(left < right) {

                longlongsum=(longlong)nums[i] + nums[j] + nums[left] + nums[right];

                if(sum == target) {

                    ans.push_back( {nums[i], nums[j], nums[left], nums[right]});
```

```

        while(left < right && nums[left] == nums[left + 1])
            left++;

        while(left < right && nums[right] == nums[right - 1])
            right--;

        left++;
        right--;
    }
    elseif(sum < target) {
        left++;
    }
    else {
        right--;
    }
}
}
}

return ans;
}
};

```

4.Output:

Saved

Ln 31, Col 23

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Input

nums =

[1,0,-1,0,-2,2]

target =

0

Output

[-2,-1,1,2], [-2,0,0,2], [-1,0,0,1]

Expected

[-2,-1,1,2], [-2,0,0,2], [-1,0,0,1]

5. Learning outcomes (What I have learnt):

1. Learned how to solve the 4Sum problem using sorting and the two-pointer technique.
2. Understood how to remove duplicate quadruplets efficiently.
3. Learned to optimize a brute-force $O(n^4)$ solution to $O(n^3)$.
4. Understood the importance of using `long long` to prevent integer overflow.
5. Improved understanding of array traversal, sorting, and pointer-based algorithms.

1.(c) LeetCode Problem: 1383.Maximum performance of a Team.

1.Aim: To design and implement an efficient algorithm to select at most k engineers such that the team's performance is maximized

2.Objective:

- 1.Toapply greedy algorithms and priority queue techniques.
- 2.Toefficientlycalculate maximum team performance using sorting and heaps.
- 3.Toanalyse time and space complexity of the solution.
- 4.Tovalidate the solution using test cases and edge cases.

4.Implementation/Code:

```
class Solution {
public:
    int maxPerformance(int n, vector<int>& speed,
                      vector<int>& efficiency, int k) {

        vector<pair<int,int>> engineers;

        for (int i = 0; i < n; i++) {
            engineers.push_back({efficiency[i], speed[i]});
        }

        sort(engineers.rbegin(),engineers.rend());

        priority_queue<int,vector<int>, greater<int>> pq;

        long long speedSum = 0;
        long long ans = 0;
        const int MOD = 1e9 + 7;

        for (auto &e : engineers) {
            int eff = e.first;
            int spd = e.second;

            if (pq.size() == k) {
                speedSum -= pq.top();
                pq.pop();
            }

            pq.push(spd);
            speedSum += spd;
        }
    }
};
```

```

        ans = max(ans, speedSum * eff);
    }

    return ans % MOD;
}
};

```

4.Output:



5.Learning outcomes (What I have learnt):

1. Learned how to solve optimization problems using the Greedy Algorithm.
2. Understood the use of Priority Queue (Min Heap) for maintaining the best k elements.
3. Learned how sorting by efficiency simplifies performance calculation.
4. Improved understanding of STL containers such as vector , pair , sort and priority_queue.
5. Learned to design an efficient solution with $O(n \log n)$ time complexity instead of checking all possible teams.